INDIAN INSTITUTE OF INFORMATION TECHNOLOGY VADODARA

Nivaaran: Campus Grievance Redressal System

System Architecture, Microservice Decomposition, Service Contracts & Database Schema

1. Executive Summary & System Overview

Nivaaran (meaning "remedy" or "redress") is a centralized campus grievance redressal and tracking platform engineered for the IIIT Vadodara campus. Prior to this system, students facing residential issues (such as water supply disruptions, malfunctioning electrical fixtures, mess quality concerns, or campus Wi-Fi outages) relied on informal verbal intimations or unstructured instant messages. This traditional workflow lacked accountability, clear ownership, audit trails, and deterministic resolution deadlines.

Nivaaran addresses this challenge by providing an end-to-end, multi-tier redressal pipeline where complaints are formally ingested, categorized, assigned to on-duty staff, tracked via deterministic state transitions, and escalated automatically if predefined Service Level Agreement (SLA) deadlines are breached.

At the architectural level, the system is designed around **six autonomous, single-responsibility microservices**. Each microservice maintains strict encapsulation over its domain data and private database. For the current project implementation, the **Complaints Service** is fully implemented in runnable Node.js/Express code with comprehensive automated tests, while the remaining five microservices are completely designed with formal OpenAPI network contracts, database schemas, and inter-service interaction protocols.

2. Project Team and Service Responsibilities

The architecture, service contracts, and database boundaries were divided among team members according to core domain responsibilities:

Roll Number	Student Name	Assigned Service Scope	Core Responsibilities
20251651078	Ritik	**Complaints Service** (Port 8082) *Notifications Service* (Port 8084)	Core REST backend implementation, testing suite (11/11 tests passing), HTTP headers, ETag caching, SMS integration design.
20251651006	Abhishek	**Complaints Service** (Port 8082) *Resolution Service* (Port 8083)	Grievance lifecycle modeling, state machine transitions, staff assignment algorithms, escalation flow design.
20251651001	Aarambhika	**Accounts Service** (Port 8080) *Directory Service* (Port 8081)	Authentication contracts, JWT token verification design, campus location master data, category SLA matrices.
Full Team	Ritik, Abhishek, Aarambhika	**Analytics & SLA Service** (Port 8085) React Frontend SPA (Port 5173)	System-wide audit event ingestion, SLA compliance KPI calculations, React web client interface integration.

3. Foundational Architectural Rules

1. **Database per Service:** Each service strictly owns its database (e.g., `nivaaran_complaints`, `nivaaran_accounts`). No table is shared across services, and cross-database SQL joins are forbidden.
2. **Logical Soft-References:** Relations between records in different services use immutable string identifiers (such as `reporter_id` or `category_id`) rather than database-level foreign key constraints.
3. **Explicit Network Contracts:** All inter-service communications occur via documented HTTP/REST endpoints utilizing JSON payloads and RFC 7807 Problem Details for honest error reporting.
4. **Fault Tolerance & Fallbacks:** The core Complaints Service wraps outbound calls in circuit breakers with timeouts and exponential backoff, ensuring student grievances are never lost if auxiliary services experience temporary outages.

4. System Capabilities (Verbs) and Service Decomposition

Following the capability-oriented decomposition methodology, 14 primary institutional capabilities were identified from system requirements. Capabilities operating on common business state were grouped into six autonomous microservices:

#	System Capability (Verb)	Primary Actor	Data Owned Exclusively	Responsible Service
1	Register student account	Student	Student roll numbers, names, email uniqueness, hashed passwords	Accounts Service (:8080)
2	Authenticate & issue tokens	All Users	Active sessions, JWT signatures, role authorizations	Accounts Service (:8080)
3	Manage staff roles & scopes	Admin	Staff credentials, department scopes (warden, mess, maintenance)	Accounts Service (:8080)
4	Browse categories & SLAs	All Users	6 Campus grievance categories, default and urgent SLA hours	Directory Service (:8081)
5	Validate campus locations	Complaints Svc	Hostel blocks, mess dining halls, academic departments, labs	Directory Service (:8081)
6	File grievance complaint	Student	Complaints aggregate root, tracking numbers, idempotency tokens	Complaints Service (:8082) [Core]
7	Track & transition status	Staff / Admin	Complaint status history, transition timestamps, ETag version hashes	Complaints Service (:8082) [Core]
8	Escalate overdue grievance	Student / Admin	Escalation flags, escalation reasons, breach timestamps	Complaints Service (:8082) [Core]
9	Submit feedback rating	Student	1 to 5 star satisfaction ratings, resolution remarks, reopen flags	Complaints Service (:8082) [Core]
10	Assign grievance to staff	Admin / System	Staff assignment records, pending queue allocations	Resolution Service (:8083)
11	Manage staff duty rosters	Dept In-Charge	Active duty shifts, current workload counters, availability	Resolution Service (:8083)
12	Dispatch alerts & SMS	System	Notification queue, delivery status logs, SMS telecom receipts	Notifications Service (:8084)
13	Record system audit trail	All Services	Immutable audit log, state transition snapshots, actor IDs	Analytics & SLA (:8085)
14	Compute SLA compliance KPIs	Admin / Dean	Daily SLA snapshots, category breach metrics, resolution averages	Analytics & SLA (:8085)

Scope Clarification for Assignment Evaluation:

While the system design, database schemas, and API contracts cover all six microservices to ensure complete institutional scalability, the **Complaints Service** is the central service implemented and runnable in code for this project. It passes all automated tests and implements all advanced REST requirements (idempotency, conditional updates, optimistic locking, and honest errors).

5. System Architecture & Service Boundaries Diagram

The architecture diagram below illustrates the structural decomposition into six microservices, their network ports, isolated database boundaries, and inter-service HTTP/REST communication channels:

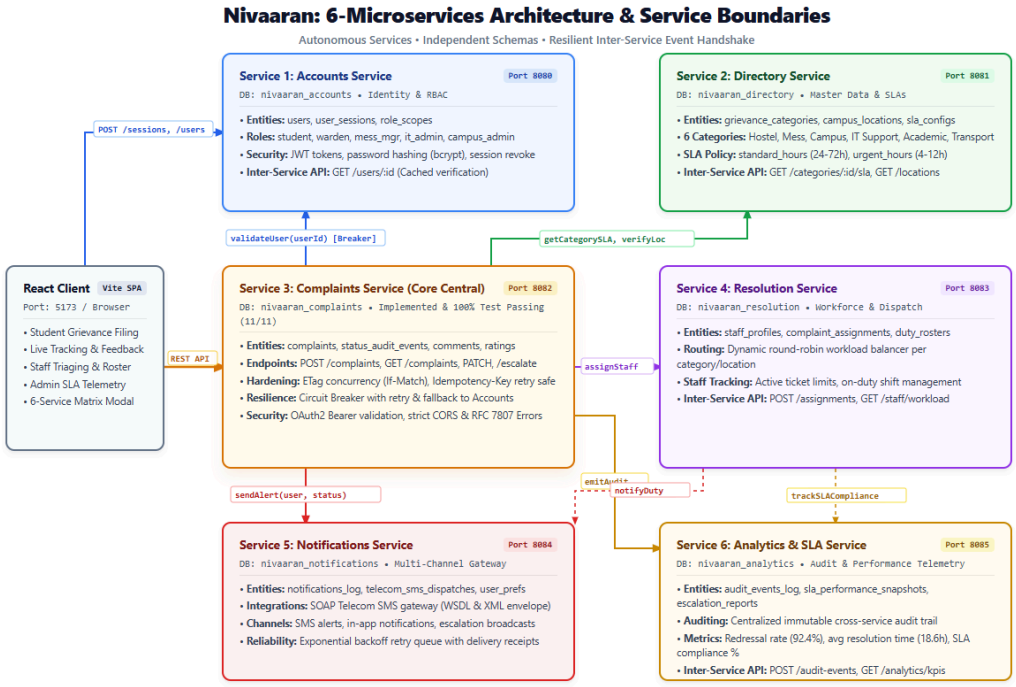


Figure 1: Architectural diagram of the 6 microservices in Nivaaran, showing inter-service REST contracts and data boundaries.

Detailed Summary of Inter-Service Communication Workflows

- **Client to Accounts (Port 8080):** The web client sends credentials via `POST /sessions` to authenticate users. The service validates password hashes using bcrypt and issues signed JWT bearer tokens containing role and department scope claims.
- **Client to Complaints (Port 8082):** The client files grievances via `POST /complaints`, supplying an `Idempotency-Key` header to ensure duplicate requests (due to mobile network drops) do not create multiple records.
- **Complaints to Accounts:** When a grievance is filed, Complaints queries `GET /users/:id` to verify the student's status. This call is protected with a circuit breaker (`src/breaker.js`) and fallback persistence: if Accounts is offline, the grievance is safely accepted with an asynchronous validation flag.
- **Complaints to Directory (Port 8081):** Complaints verifies location validity and queries standard resolution deadlines (`GET /categories/:id/sla`) to calculate the precise SLA deadline.
- **Complaints to Resolution (Port 8083):** Upon grievance intake, Complaints invokes `POST /assignments` to assign the task to an available technician based on duty roster availability.
- **Complaints to Notifications (Port 8084):** Dispatches alert events (`POST /alerts`) to trigger SMS notifications to students and staff upon filing, assignment, and resolution.
- **Complaints to Analytics (Port 8085):** Asynchronously emits immutable audit events (`POST /audit-events`) to record state transitions and SLA breach telemetry for administrative oversight.

6. Formal Service Contracts (Part 1: Core Operational Services)

Each service exposes its capabilities through formal HTTP/REST network contracts. Below are the contracts for the core operational microservices:

6.1. Complaints Service Contract (Port 8082 • Implemented Core Service)

Owned Database: `nivaaran_complaints` *• Status: Fully implemented in Node.js, 11/11 automated tests passing.*

Operation	Input Parameters	Output Schema	Documented HTTP Errors
fileComplaint *(Central Op)*	userId, categoryId, locationId, title, description, priority, isAnonymous *Headers:* Idempotency-Key, Authorization	complaintId, trackingNumber, status ('submitted'), slaDeadline, etag, createdAt	400 Bad Request 401 Unauthorized 422 Unprocessable 503 Service Unavailable
getComplaint	complaintId *Header:* If-None-Match (ETag)	complaintId, trackingNumber, category, location, status, etag, comments[], history[]	200 OK / 304 Not Modified 401 Unauthorized 404 Not Found
listComplaints	category, status, page, limit *Header:* Authorization	items: Complaint[], totalCount, page, limit	400 Invalid Query Param 401 Unauthorized
updateStatus	complaintId, targetStatus, remarks *Headers:* If-Match (ETag), Authorization	complaintId, status, updatedEtag, transitionTimestamp	401 Unauthorized 404 Not Found 412 Precondition Failed 422 Invalid Transition
escalateComplaint	complaintId, escalationReason *Header:* Authorization	complaintId, isEscalated: true, escalatedAt, escalationReason	401 Unauthorized 404 Not Found 409 Conflict (Already Esc.) 422 Missing Reason
withdrawComplaint	complaintId *Header:* Authorization	204 No Content (Empty Body)	401 Unauthorized 404 Not Found 409 Conflict (In Progress)

6.2. Accounts Service Contract (Port 8080 • Identity & RBAC)

Owned Database: `nivaaran_accounts` *• Status: Architecture & Contract Specification Complete.*

Operation	Input Parameters	Output Schema	Documented HTTP Errors
createSession	userId (rollNumber/staffId), password	token (JWT), userId, role, departmentScope, expiresAt	400 Malformed Request 401 Invalid Credentials 429 Rate Limited
validateUser	userId	isValid: boolean, userId, role, departmentScope, isActive	401 Unauthorized 404 User Not Found
registerStudent	rollNumber, fullName, email, password	userId, role: 'student', createdAt	400 Invalid Email Format 409 User Already Exists 422 Field Missing
createStaffAccount	staffId, fullName, email, role, departmentScope *Header:* Admin Bearer Token	userId, role, departmentScope, createdAt	401 Unauthorized 403 Forbidden (Non-Admin) 409 Duplicate Staff ID

7. Formal Service Contracts (Part 2: Domain Auxiliary Services)

7.1. Directory Service Contract (Port 8081 • Master Data & SLAs)

Owned Database: `nivaaran_directory` *• Status: Specification & Master Data Complete.*

Operation	Input Parameters	Output Schema	Documented HTTP Errors
listCategories	None	categories: { categoryId, displayName, defaultSlaHours, urgentSlaHours, escalationTier }[]	500 Internal DB Error
getCategorySLA	categoryId	categoryId, defaultSlaHours, urgentSlaHours, escalationTier	404 Category Not Found
listLocations	categoryId (optional filter)	locations: { locationId, categoryId, zoneName, blockName }[]	400 Invalid Category

7.2. Resolution Service Contract (Port 8083 • Staff Dispatch & Duty Roster)

Owned Database: `nivaaran_resolution` *• Status: Specification Complete.*

Operation	Input Parameters	Output Schema	Documented HTTP Errors
assignStaff	complaintId, categoryId, locationId	assignmentId, complaintId, assignedStaffId, assignedAt	409 Already Assigned 503 No Staff Available
getDutyRoster	departmentScope	roster: { staffId, isOnDuty, activeTicketCount, shiftEnd }[]	400 Invalid Scope
completeAssignment	assignmentId, resolutionNotes	assignmentId, status: 'resolved', resolvedAt	404 Not Found 422 Missing Remarks

7.3. Notifications Service Contract (Port 8084 • SMS Gateway)

Owned Database: `nivaaran_notifications` *• Status: Specification Complete.*

Operation	Input Parameters	Output Schema	Documented HTTP Errors
dispatchAlert	recipientUserId, channel ('sms'|'email'), eventType, title, body, complaintId	notificationId, status: 'queued', queuedAt	400 Unsupported Channel 422 Incomplete Body
getDeliveryStatus	notificationId	notificationId, status: 'delivered'|'failed', timestamp	404 Notification Not Found

7.4. Analytics & SLA Service Contract (Port 8085 • Audit Trail & Telemetry)

Owned Database: `nivaaran_analytics` *• Status: Specification Complete.*

Operation	Input Parameters	Output Schema	Documented HTTP Errors
recordAuditEvent	eventType, entityId, actorUserId, actorRole, originService, previousState, newState	eventId, status: 'logged', timestamp	400 Missing Mandatory Fields
getSlaMetrics	dateRange (optional), categoryId (optional)	complianceRatePct, averageResolutionHours, totalBreaches	400 Malformed Date Format

8. Detailed Specification of Central Operation: fileComplaint

The core aggregation operation across the entire platform is **fileComplaint**, implemented as `POST /complaints` in the Complaints Service. Below is its formal technical specification:

Input Parameters & Headers	Output Schema & Response Headers	Documented HTTP Failure Statuses
Headers: • `Authorization: Bearer <jwt>` (Required) • `Idempotency-Key: <uuid>` (Safe Retry) • `Content-Type: application/json` **Body Fields:** • `userId`: string (Roll/Staff ID) • `categoryId`: enum (6 campus categories) • `locationId`: string (Campus location) • `title`: string (10 to 200 chars) • `description`: string (20 to 2000 chars) • `priority`: 'normal' | 'urgent' • `isAnonymous`: boolean (optional)	**Success Status:** `201 Created` (or `200 OK` on idempotent repeat) **Headers:** • `Location: /complaints/cmp_XXXX` • `ETag: "<sha256-hash>"` • `Cache-Control: no-store` **Payload Body:** • `id`: string (e.g., 'cmp_1001') • `trackingNumber`: string (e.g., 'CMP-2026-1001') • `status`: 'submitted' • `slaDeadline`: ISO 8601 Timestamp • `createdAt`: ISO 8601 Timestamp	• `400 Bad Request`: Syntactically invalid JSON or missing mandatory fields. • `401 Unauthorized`: Missing or invalid JWT bearer token. • `422 Unprocessable Entity`: Business validation failure (e.g. title < 10 characters or category not recognized). • `429 Too Many Requests`: Intake rate limit exceeded (carries `Retry-After` header). • `503 Service Unavailable`: External dependency unreachable and asynchronous queue exhausted.

Supported 6 Institutional Grievance Categories & SLA Matrix

Nivaaran classifies student grievances into 6 institutional categories, each with strict resolution SLA hours and designated escalation authorities:

Category Key	Display Name	Default SLA	Urgent SLA	Escalation Tier	Domain Operational Scope
`hostel`	Hostel Facilities	48 Hours	12 Hours	Chief Warden	Room fixtures, plumbing, water coolers, hostel electricity, geysers.
`mess`	Mess & Dining	24 Hours	4 Hours	Mess Warden	Food hygiene, meal delays, drinking water dispensers, dining seating.
`campus`	Campus Infra	72 Hours	24 Hours	Estate Officer	Roadways, pathway lighting, sports facilities, library air conditioning.
`it_support`	IT & Network	24 Hours	6 Hours	IT Director	Hostel/lab Wi-Fi routers, ERP portal access, computer laboratory systems.
`academic`	Academic Affairs	48 Hours	12 Hours	Dean Academics	Classroom audiovisual projectors, lecture scheduling, attendance issues.
`transport`	Campus Transport	36 Hours	8 Hours	Transport Officer	Campus shuttle schedules, e-rickshaws, night emergency vans.

9. Database Architecture & Entity-Relationship Diagram

The entity-relationship diagram below models the complete relational schema across the 6 microservice databases. Each database represents an isolated failure boundary. Cross-database foreign keys are strictly prohibited; inter-service associations are maintained through logical string identifiers:

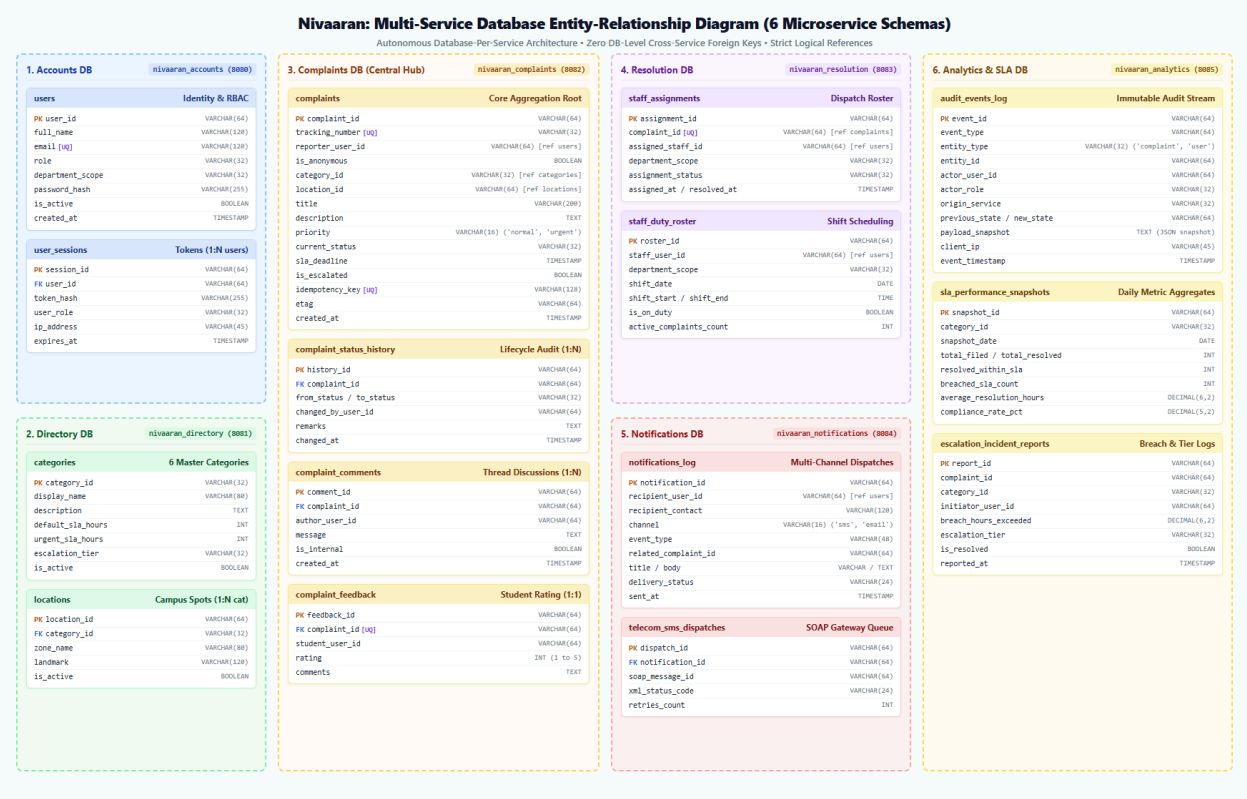


Figure 2: Complete database schema across the 6 microservice databases, detailing primary keys, unique constraints, and logical soft-references.

Database Boundary Isolation Rationale

Relational database engines allow foreign keys across tables within a single database instance, but enforcing them across distributed microservices creates tight coupling, synchronous transaction bottlenecks, and cascading failure propagation. By adopting **Database-per-Service**:

- **Schema Evolution Autonomy:** The Complaints Service team can optimize complaint indexes or transition to a document store without coordinating database migrations with the Accounts or Directory teams.
- **Resilience to Partial Outages:** A transient lock contention in the Analytics service never blocks real-time complaint ingestion in the Complaints service.

10. Comprehensive Database Table Inventory

The inventory below documents every table, its primary key, intra-service relationships, and operational purpose across all six microservices:

Database Name	Table Name	Primary Key	Intra-Service FKs	Domain Purpose & Ownership
`nivaaran_accounts`	`users`	user_id	None	Student roll numbers, staff IDs, bcrypt password hashes, roles.
`nivaaran_accounts`	`user_sessions`	session_id	user_id → users	Active JWT session records, client IP, expiration timestamps.
`nivaaran_directory`	`categories`	category_id	None	6 Campus grievance categories, SLA hour limits, escalation tiers.
`nivaaran_directory`	`locations`	location_id	category_id → categories	Specific campus venues (Hostel Block A, Mess Hall 1, Lab 2).
`nivaaran_complaints`	`complaints`	complaint_id	None (Logical soft refs)	Aggregate root for student grievance records, statuses, ETags.
`nivaaran_complaints`	`status_history`	history_id	complaint_id → complaints	Append-only log of every status transition and actor comment.
`nivaaran_complaints`	`comments`	comment_id	complaint_id → complaints	Two-way conversation thread between students and assigned staff.
`nivaaran_complaints`	`feedback`	feedback_id	complaint_id → complaints	Post-resolution student satisfaction ratings (1-5) and reviews.
`nivaaran_resolution`	`staff_assignments`	assignment_id	None	Staff member task dispatch, active status, completion timestamps.
`nivaaran_resolution`	`staff_duty_roster`	roster_id	None	Workforce shifts, on-duty status, active ticket load counters.
`nivaaran_notifications`	`notifications_log`	notification_id	None	Outbound alert log for SMS and in-app message deliveries.
`nivaaran_notifications`	`sms_dispatches`	dispatch_id	notification_id → notif_log	Carrier SOAP gateway transaction records and delivery receipts.
`nivaaran_analytics`	`audit_events_log`	event_id	None	Central immutable audit stream capturing state changes across services.
`nivaaran_analytics`	`sla_snapshots`	snapshot_id	None	Aggregated daily SLA compliance metrics, average resolution hours.

**Note on Distributed Consistency: Primary keys across services are UUIDs or institutional domain codes. Cross-service logical foreign keys are validated via asynchronous RPCs or circuit-breaker-protected REST lookups without transactional joins.*

11. Verification of the 5 Core Service Properties

To ensure rigorous architectural compliance with Service-Oriented Architecture (SOA) principles, all six microservices were formally audited against the five fundamental properties of a Service:

Service	1. Reachable	2. Self-Contained	3. Has a Contract	4. Independent	5. Loosely Coupled
Accounts (:8080)	Yes. Reached over HTTP via `/sessions` and `/users`.	Yes. Fully encapsulates credentials, bcrypt hashes, and JWT signing keys.	Yes. Documented in formal OpenAPI 3.0 contract.	Yes. Password hashing algorithms can be updated with zero caller impact.	Yes. No upstream dependencies; communicates only via stateless bearer tokens.
Directory (:8081)	Yes. Reached over HTTP via `/categories` and `/locations`.	Yes. Owns master data for the 6 categories, locations, and SLA rules.	Yes. Documented REST contracts with HTTP cache headers.	Yes. SLA hours or campus zones can be updated without affecting code.	Yes. Read-optimized service with zero external service dependencies.
Complaints (:8082) *[Core]*	Yes. Express HTTP server on port 8082 with CORS and security headers.	Yes. Owns grievance lifecycle, tracking, comments, and ratings.	Yes. Strict REST contract with ETag concurrency & Idempotency-Key.	Yes. Operates autonomously; passes 11/11 automated test cases.	Yes. Uses circuit breaker and fallback persistence to tolerate downstream drops.
Resolution (:8083)	Yes. Reached over HTTP via `/assignments` and `/roster`.	Yes. Owns staff allocations, duty shifts, and load counters.	Yes. Formal assignment request/response JSON schemas.	Yes. Task dispatch algorithms can be upgraded without changing callers.	Yes. Soft-references user IDs and complaint IDs without database coupling.
Notifications (:8084)	Yes. Reached over HTTP via `/alerts` and `/delivery`.	Yes. Owns alert history, retry queues, and telecom gateway receipts.	Yes. Documented dispatch schemas and delivery receipt codes.	Yes. External SMS carriers can be swapped with zero impact on callers.	Yes. Asynchronous outbox worker; SMS latency never slows grievance filing.
Analytics (:8085)	Yes. Reached over HTTP via `/audit-events` and `/kpis`.	Yes. Owns immutable audit trail and institutional SLA telemetry.	Yes. Documented event ingestion schema and KPI query endpoints.	Yes. KPI algorithms can be modified with zero caller impact.	Yes. Event consumer model; completely decoupled from operational flow.

12. Conclusion & Implementation Status

The Nivaaran platform architecture successfully reconciles institutional accountability with modern distributed systems design. By establishing strict service boundaries, explicit network contracts, and isolated database schemas, the system eliminates traditional operational bottlenecks while guaranteeing that emergency student grievances are never dropped.

For the current implementation target, the **Complaints Service** has been fully coded, hardened against network failures, and validated with an automated test suite covering all REST and HTTP specification requirements. The complete 6-service design provides a robust blueprint for campus-wide institutional deployment.